Comment jouer au Sokoban

Guide du joueur — Projet Sokoban POO L2 S4

Fonctionnalités implémentées

Étape	Description	Statut
1	Organisation en packages	✅
2-3	Hiérarchie des éléments	✅
4	Classe Carte	✅
5-7	Mode texte basique + toString	✅
8-10	Lecture depuis fichier	✅
11	Enum Direction avec dx/dy	✅
12	Classe Robot	✅
13-15	Déplacements + finDePartie (stub→réel)	✅
16-17	ModeTexte + SokobanTexte	✅
18	Fin de partie (détection caisses/destinations)	✅
19	VueSokoban – affichage Swing avec images GIF	✅
20	KeyListener – flèches directionnelles	✅
21	Images orientées du robot selon Direction	✅
22	Compteur de mouvements	✅
23	Undo – annulation du dernier mouvement	✅
24	Restart – redémarrage du niveau	✅
25	Changement de niveau automatique	✅

Générer la documentation pour tous les packages

```bash
javadoc -d doc -sourcepath src -subpackages modele:vueTexte:VueGraphique -encoding UTF-8 -charset UTF-8 -docencoding UTF-8
```

Compilation

Depuis la racine du projet (`sokoban/`) :

```bash
javac -d bin -sourcepath src src/modele/*.java src/vueTexte/*.java src/VueGraphique/*.java
```

Lancement

Mode texte

```bash
java -cp bin vueTexte.SokobanTexte
```

Commandes : `z`=haut · `s`=bas · `q`=gauche · `d`=droite · `a`=undo · `e`=restart · `x`=quitter

Mode graphique

```bash
java -cp bin VueGraphique.Sokoban
```

Commandes : flèches directionnelles · `U`=undo · `R`=restart

Création des JARs (dépôt final)

```bash
# JAR graphique
jar cfe sokobanSalehMahamatSaleh.jar VueGraphique.Sokoban -C bin . -C . map -C . images

# JAR texte
jar cfe sokobanTexteSalehMahamatSaleh.jar vueTexte.SokobanTexte -C bin . -C . map

# Lancement
java -jar sokobanSalehMahamatSaleh.jar

java -jar sokobanTexteSalehMahamatSaleh.jar
```

1. Contrôles

Utilisez les touches suivantes pour jouer :

Touche	Action
Flèche Haut	Déplacer le robot vers le haut
Flèche Bas	Déplacer le robot vers le bas
Flèche Gauche	Déplacer le robot vers la gauche
Flèche Droite	Déplacer le robot vers la droite

U	Annuler le dernier mouvement (Undo)
R	Recommencer le niveau (Restart)

2. Règles du jeu

1. Vous contrôlez le robot (le personnage qui se déplace).
2. Poussez les caisses (marron) sur les emplacements prévus (cases marquées par un point).
3. Attention : vous pouvez seulement pousser une caisse, pas la tirer.
4. Attention : vous ne pouvez pousser qu'une seule caisse à la fois — si deux caisses sont collées, le mouvement est impossible.
5. La partie est gagnée quand toutes les caisses sont sur une destination.

3. Les éléments visuels

Chaque case de la carte a une signification précise :

Élément	Signification
Mur (brique) #	Infranchissable — le robot ne peut pas passer
Sol (fond clair) espace	Case vide où le robot peut marcher
Destination .	Emplacement où une caisse doit aller
Caisse marron $	Caisse à pousser
Caisse verte *	Caisse correctement placée sur une destination
Robot @ / +	Votre personnage — son image change selon la direction

4. Déroulement d'une partie

```
Lancer le jeu
     ↓
Utiliser les flèches pour déplacer le robot
     ↓
Pousser toutes les caisses sur les destinations
     ↓
Niveau gagné → passage automatique au niveau suivant
     ↓
Niveaux : 1 → 2 → 3
```

5. Conseils

- Si vous êtes bloqué → appuyez sur U pour annuler le dernier mouvement.

- Si la situation est trop mauvaise → appuyez sur R pour recommencer depuis le début.
- Vous pouvez aussi changer de niveau manuellement via le menu déroulant en haut de la fenêtre.

7.Difficultés rencontrées et choix effectués

• Représentation des occupants : stocker robot et caisses directement comme booléens sur les
éléments fixes (plutôt que des listes séparées) simplifie les accès et les mises à jour lors des déplacements.

• Caractères d'affichage composites : la méthode toString() d'Element centralise la logique de
composition (@, +, $, *) ; cela évite de la dupliquer dans la vue texte.

• Largeur variable des lignes : certaines cartes ont des lignes de longueurs inégales. Le constructeur de Carte prend la largeur maximale et complète les lignes courtes avec du vide.

• Enum Direction avec dx/dy : ce choix élimine tout switch dans la logique de déplacement, rendant le code plus concis et extensible.